

Linux Memory- Mapped File Management System

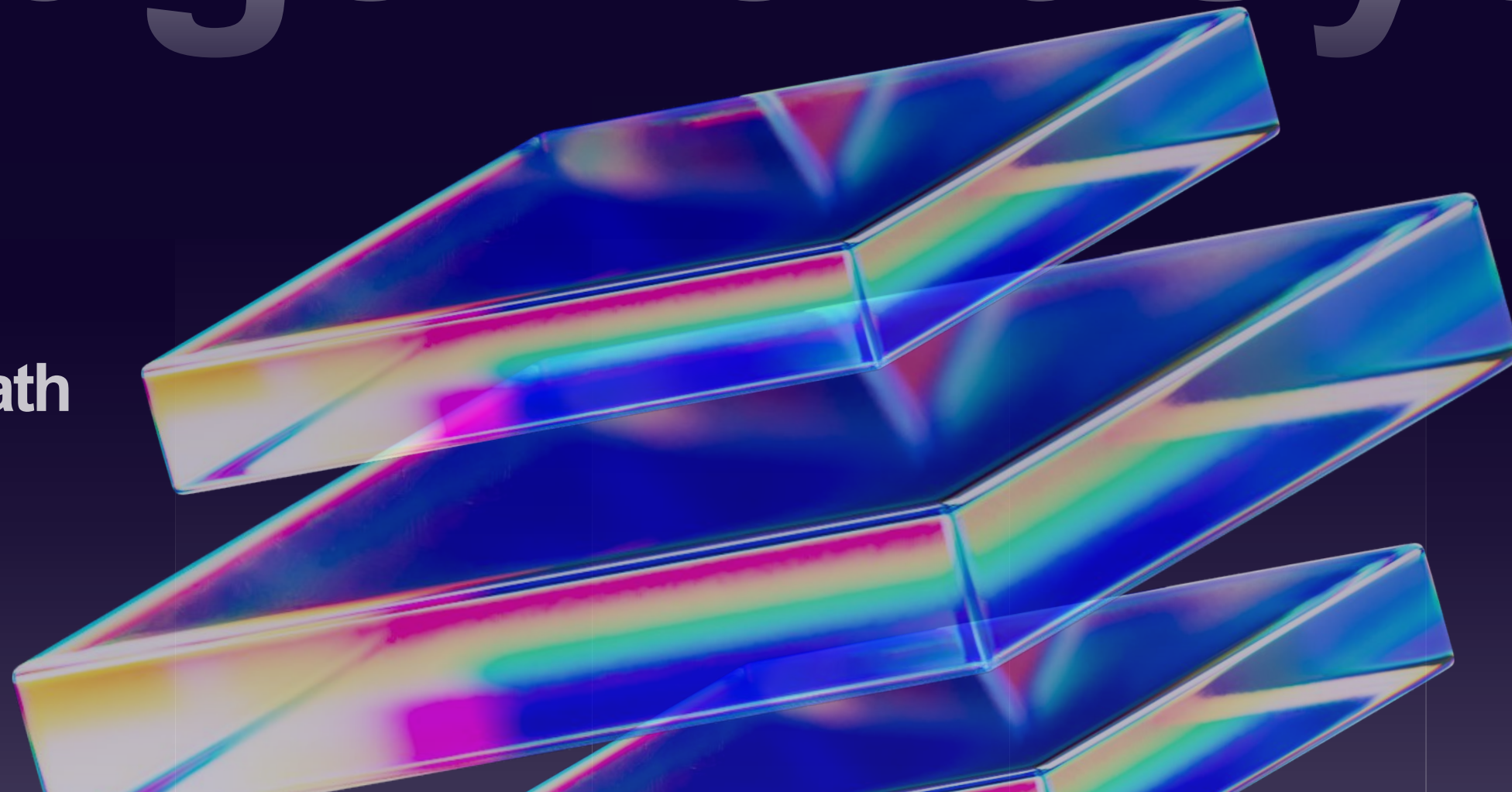
TEAM

2520090128 - B. Bharath

2520089071 - Harsha

2520090089 - P. Uday

OSSP sec - 11



Introduction to Problem statement

Traditional file I/O uses `read()` and `write()` system calls to transfer data between kernel space and user space.

For repeated file access, this involves system-call overhead and data movement between buffers.

We developed a Linux-based tool that uses memory-mapped file I/O through the `mmap()` system call.

Instead of repeatedly copying file data using `read()/write()`, the file is mapped into the process's virtual address space and accessed through memory.

Traditional I/O

File → Kernel Buffer → User Buffer

Memory-Mapped I/O

File → Virtual Memory Mapping → Process

The project documentation identifies comparison of `mmap()` with traditional `read()/write()` as one of its primary goals.

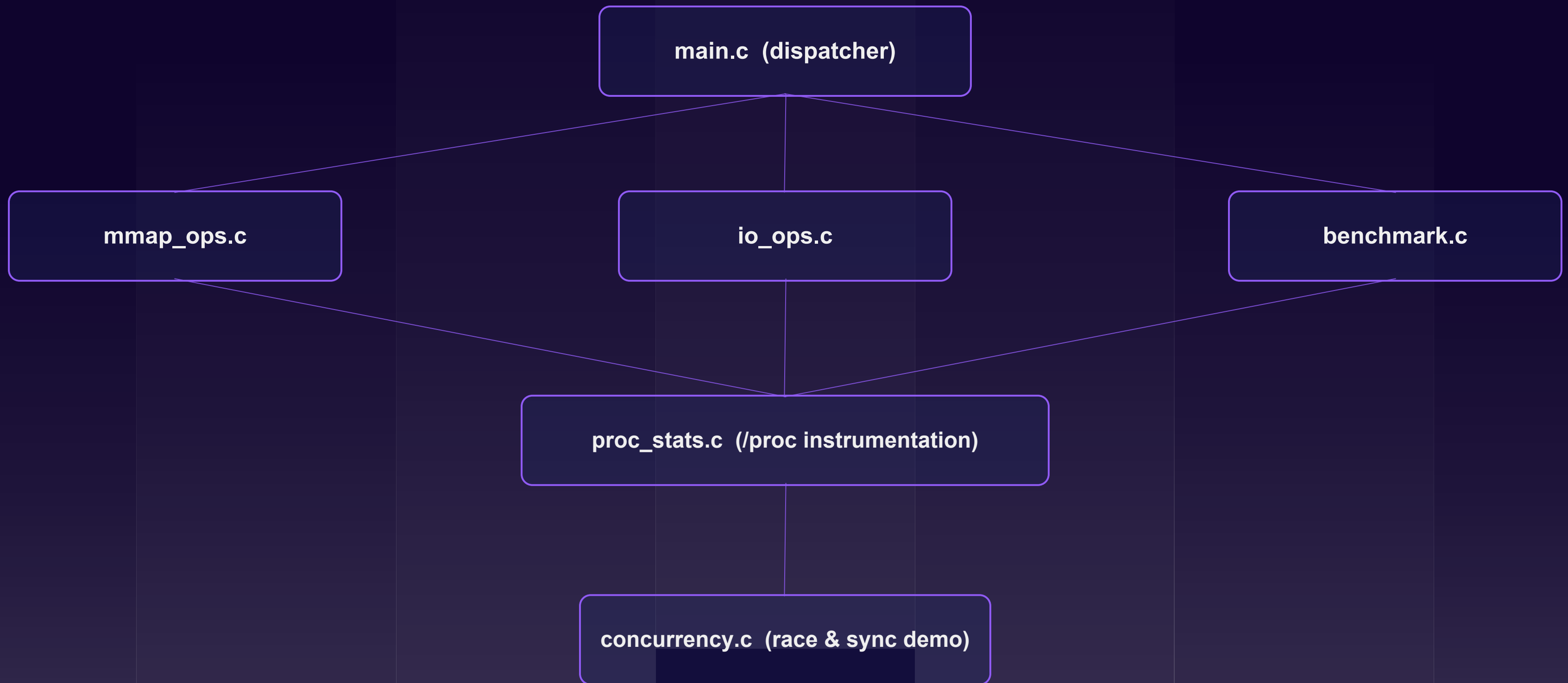
OBJECTIVES

IMPLEMENT MEMORY-MAPPED FILE OPERATIONS USING:MMAP()

- `munmap()`
- `msync()`
Compare `mmap()` with traditional `read()/write()` operations.
- Demonstrate:
- `MAP_PRIVATE`
- `MAP_SHARED`

- **ANALYZE MINOR AND MAJOR PAGE FAULTS.**
 - Demonstrate inter-process communication using shared memory.
- Demonstrate race conditions between processes.
- Implement synchronization using POSIX semaphores.
- Inspect process memory using `/proc`.
 - Trace system calls using `strace`.

System Architecture



CO wise implementation

CO - 1

IMPLEMENTED THROUGH:

- **LINUX SYSTEM CALLS**
- **USER-SPACE APPLICATION INTERACTING WITH KERNEL SERVICES**
- **OPEN(), MMAP(), munmap(), msync()**
- **STRACE To observe system-call execution**
- **/proc for system-level inspection**

CONCEPT Demonstrated:

Application → System Call → Kernel → Hardware/Storage

CO wise implementation

CO - 2

IMPLEMENTED THROUGH:

- **FORK()**
- **PARENT AND CHILD PROCESSES**
- **PRocess execution**
- **WAIT() / process completion**
- **Process statistics using /proc**

CONCEPT Demonstrated:

PROcess creation → Execution → Synchronization → Termination

CO wise implementation

CO - 3

IMPLEMENTED THROUGH:

- **SHARED MEMORY USING MAP_SHARED**
- **MULTIPLE PROCESSES ACCESSING SHARED DATA**
- **PROCESS synchronization**
- **POSIX semaphore**

CONCEPT Demonstrated:

Process 1 ↔ Shared Memory ↔ Process 2

IMPLEMENTATION

How the System Works

Memory-Mapped Read

- Open the file using `open()`.
- Determine file size using `fstat()`.
- Map the file using `mmap()`.
- Access the file through the returned pointer.
- Process the data.
- Unmap using `munmap()`.
- Close the file descriptor.

Memory-Mapped Write

- Open/create the file.
- Set file size using `ftruncate()`.
- Create a `MAP_SHARED` mapping.
- Write data using memory operations.
- Call `msync()` to synchronize changes.
- Unmap the region.
- Close the file.

PAGE FAULTS & VIRTUAL MEMORY

Page Fault Analysis

Minor Page Fault

- Page is already available in memory.
- The page is not yet mapped into the process.
- Does not require disk I/O.
- Relatively inexpensive.
- mmap() produces more minor faults because of demand paging.

Major Page Fault

- Page must be loaded using disk I/O.
- More expensive than a minor page fault.

Our Observation

- For the tested environment:
- mmap() generated more minor page faults.
- Minor faults increased approximately with file size.
- Major faults were zero because the test data was already cached.

Performance Analysis

mmap() vs read()

File Size	read() MB/s	mmap() MB/s
1 MB	521.17	555.83
10 MB	546.09	586.12
50 MB	545.79	571.62

Observations

- mmap() achieved approximately 6–7% higher throughput in the tested environment.
- mmap() generated more minor page faults.
- read() remained around 4 minor faults in the tests.
- Major page faults were zero.

Why mmap() Can Be Faster

- Reduced system-call overhead.
- Direct access through mapped memory.
- Kernel page-cache integration.
- Demand paging.

Concurrency — Race Condition vs Synchronized Access

RACE CONDITION

- No locking — `sched_yield()` between read & write
- Processes overwrite each other's updates
- Result: $\approx 100,000$ (lost updates)

SYNCHRONIZED (SEMAPHORE)

- `sem_wait()/sem_post()` guard the critical section
- Only one process updates at a time
- Result: 200,000 — always correct

Core Concepts — Page Fault Behavior

MINOR PAGE FAULTS

- Page is in memory, not yet mapped to process
- No disk I/O required — inexpensive
- `mmap()` generates far more of these (demand paging)

MAJOR PAGE FAULTS

- Require disk I/O to load the page
- More expensive than minor faults
- Zero for both methods once data is cached

Concurrency — Race Condition vs Synchronized Access

RACE CONDITION

- No locking — `sched_yield()` between read & write
- Processes overwrite each other's updates
- Result: $\approx 100,000$ (lost updates)

SYNCHRONIZED (SEMAPHORE)

- `sem_wait()/sem_post()` guard the critical section
- Only one process updates at a time
- Result: 200,000 — always correct

CONCLUSION

The project successfully demonstrates how Linux manages file data through virtual memory.

WE IMPLEMENTED:

- MEMORY-MAPPED FILE I/O USING MMAP()
- TRADITIONAL FILE I/O USING READ()/WRITE()
- MAP_PRIVATE AND MAP_SHARED
- PAGE-Fault analysis
- PERFORmance benchmarking
- SHared-memory IPC
- Race-condition demonstration
- Semaphore-based synchronization
- /proc process inspection
- strace system-call tracing



THANK YOU

